

Introdução ao versionamento de código com Git

Prof. Giliard Godoi

giliard.godoi@fatecourinhos.edu.br

Quem sou eu?

Prof. Giliard

- Licenciado em Matemática (UENP)
- Tecnologia em Análise e Desenvolvimento de Sistemas (UTFPR-CP)
- Mestrado em Informática Aplicada (UTFPR-CP)
- Doutorando em Ciência da Computação (USP-ICMC)
- Professor na FATEC Ourinhos desde abril 2026



Para quem é esse curso?

- ➔ Eu não sei nada sobre Git? O que ele come ou onde vive?
- ➔ Eu não sei a diferença entre Git e Github?
- ➔ Tenho medo do terminal, mas quero aprender...
- ➔ Eu quero saber mais sobre versionamento de código, por que sim!

Objetivos

- ➔ Ensinar os principais **conceitos** e **comandos** da ferramenta Git
- ➔ Fazer o seu primeiro **commit**
- ➔ Fazer o seu primeiro **push**

Além disso, não se preocupe em memorizar comandos (para isso temos uma colinha) mas em aprender conceitos.

Agenda

- O que é Git?
- Download e instalação
- Configurações iniciais
- Criando Repositório Local
- Rastrear mudanças no repositório local (História dos 3 porquinhos)
- Repositórios remotos no GitHub
- Sincronizando mudanças entre os repositórios local e remoto

O que é Git?

Git é uma ferramenta de software que serve, entre outras coisas, para manter um **histórico das modificações do código** de um projeto e **facilitar o gerenciamento** e o desenvolvimento colaborativo de software.

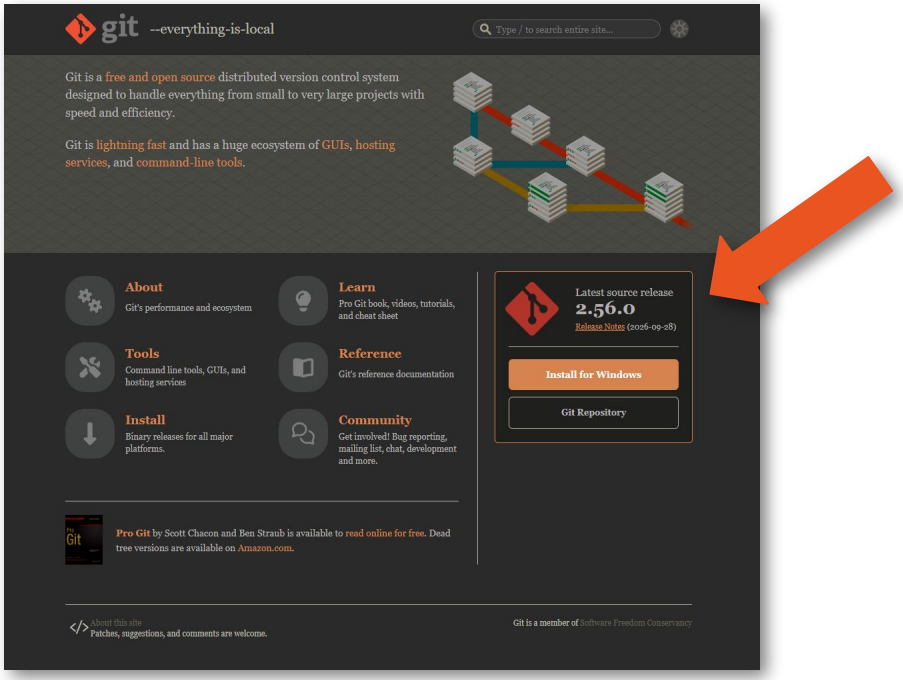
O Git começou a ser desenvolvido por **Linus Torvalds** em 2005 para versionamento do kernel do Linux, após controvérsias com a proprietária da ferramenta de versionamento utilizada desde 2002.



Git - A Short History of Git

Download e instalação

1. Visite <https://git-scm.com/>
2. Baixe o instalador para o seu sistema operacional
3. Siga as recomendações de instalação
4. Por sorte o instalador vai configurar o git para o PATH do seu sistema operacional

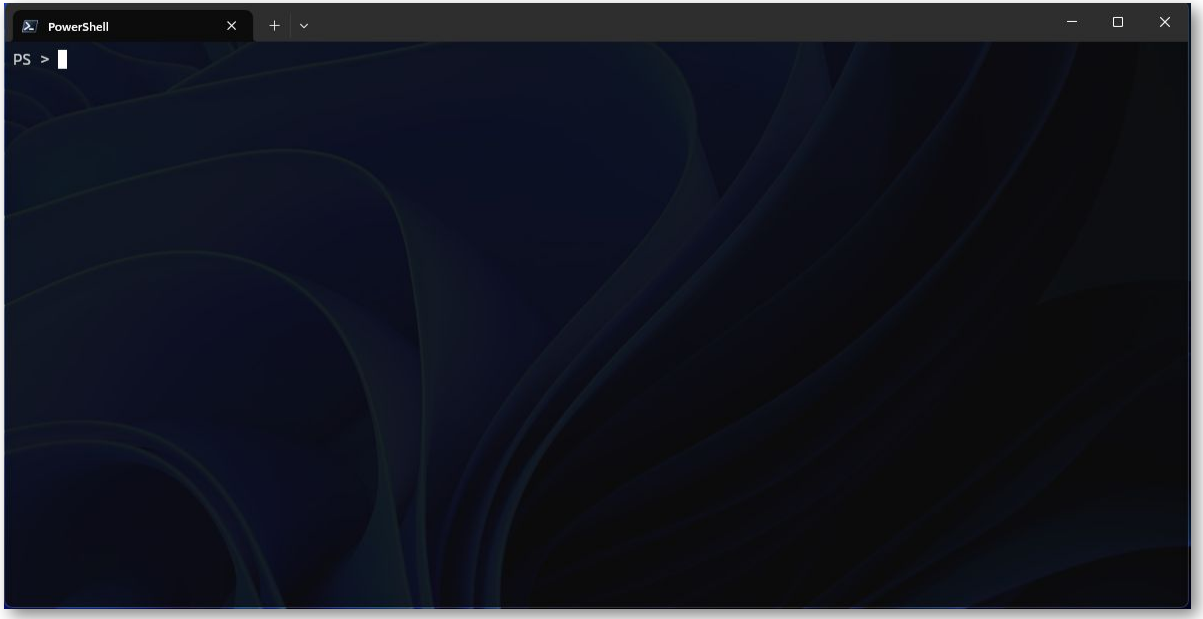




Calma!
Onde eu aperto para começar a usar o git?

Windows Terminal

Terminal com esteróides para ambiente Windows.

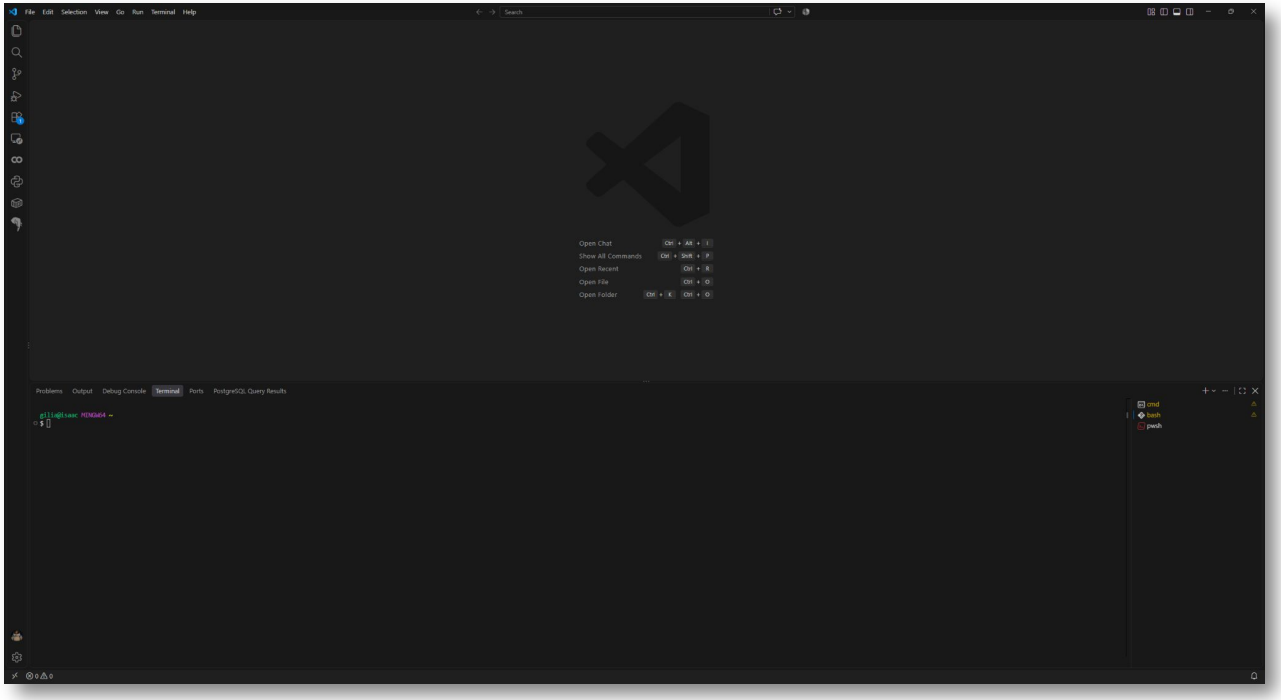


Windows Terminal installation | Microsoft Learn

Terminal integrado do VSCode

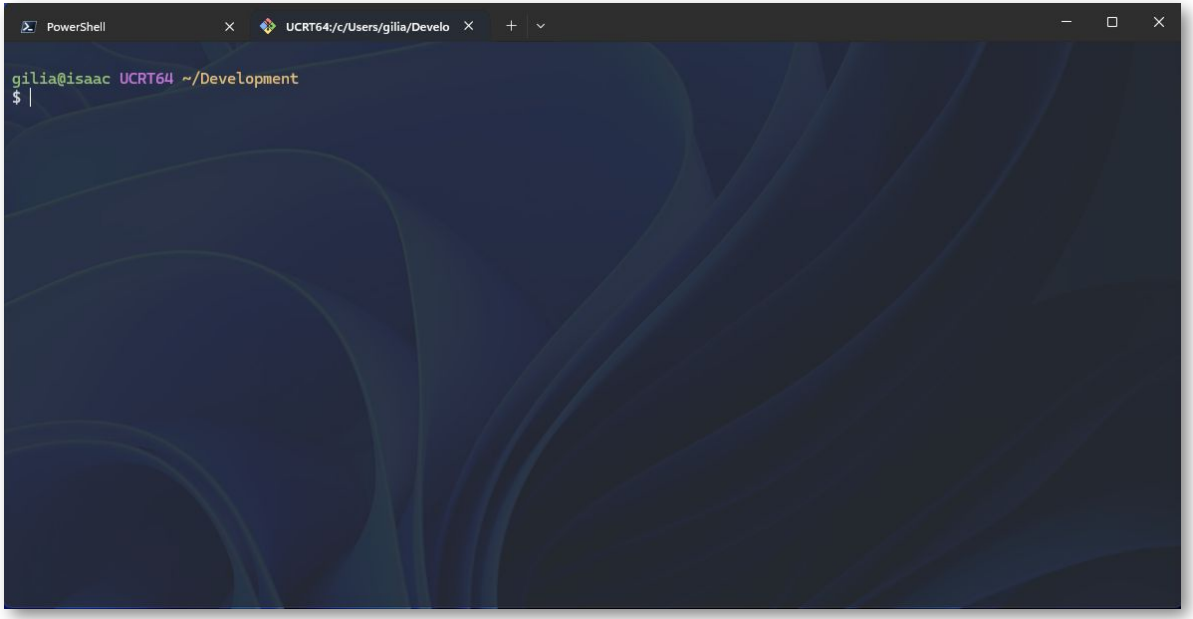
No **VSCode**

`Ctrl + ’`



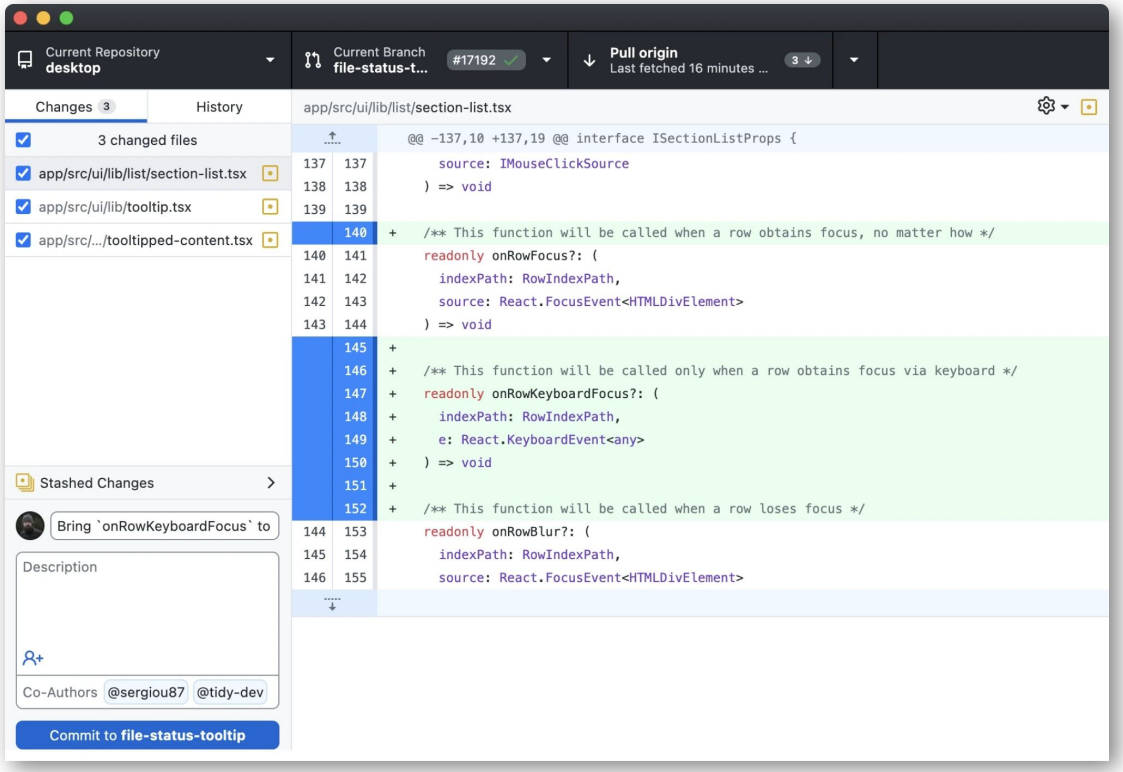
Git bash

- Você pode escolher instalar junto com o git.



GitHub Desktop

➔ Não é o foco deste minicurso



GitHub Desktop | Colaboração simples no seu desktop

Verificando a instalação

Depois da instalação nós podemos verificar se o git está acessível no terminal com o seguinte comando:

```
git --version
```



Configurações iniciais

Configurações iniciais

Antes de começarmos a usar o git nós devemos configurar três parâmetros importantes:

1. O nome do usuário
2. O email do usuário
3. E um editor de texto padrão

Algumas configurações são mantidas em um arquivo denominado `.gitconfig`

```
git config --list --show-origin
```

Exibindo as configurações

Na primeira instalação do git em sua máquina, talvez o comando para listar as configurações talvez não funcione simplesmente porque esse arquivo ainda não foi criado.

Caso isso aconteça, tente os passos dos próximos slides antes de listar as configurações do arquivo `.gitconfig`

```
git config --list --show-origin
```

Configurando o usuário

Cada “mudança” no histórico do git fica associado a um usuário. Por isso é importante registrar o nosso usuário nas configurações globais.

```
git config --global user.name “Giliard Godoi”
```

```
git config --global user.email giliard.godoi@fatecourinhos.edu.br
```

Configurando um editor

Em alguns momentos o git vai nos pedir para editar algumas mensagens de texto.

Por isso o git quer saber qual o **editor de texto puro** que ele vai chamar para que nós possamos editar essas mensagens.

```
git config --global core.editor notepad
```

Configurando um editor

Podemos configurar o **editor texto** que nós quisermos, desde que ele esteja acessível através do terminal

```
git config --global core.editor notepad

git config --global core.editor vim

git config --global core.editor “code --wait”
```



Editando o arquivo de configurações

Inclusive conseguimos editar o arquivo de configurações através do editor configurado anteriormente.

```
git config --global --edit
```

```
git config --global -e
```

Atividade para casa…

Como eu poderia configurar o nome e email para um usuário em um repositório local?

Atividade para casa…

Como eu poderia configurar um nome e email de usuário específico para todo um subdiretório de projetos?

Por exemplo, imagine que você tivesse dois diretórios:

```
faculdade/

trabalho/
```



Repositório local

Repositório local

Comumente, os arquivos e códigos de um projeto de desenvolvimento de software são mantidos em um **diretório**.

Um diretório se torna um repositório local quando inicializamos as **estruturas básicas para o rastreamento das versões** dos códigos dos programas.

Os comandos para **inicializar um repositório local** são:

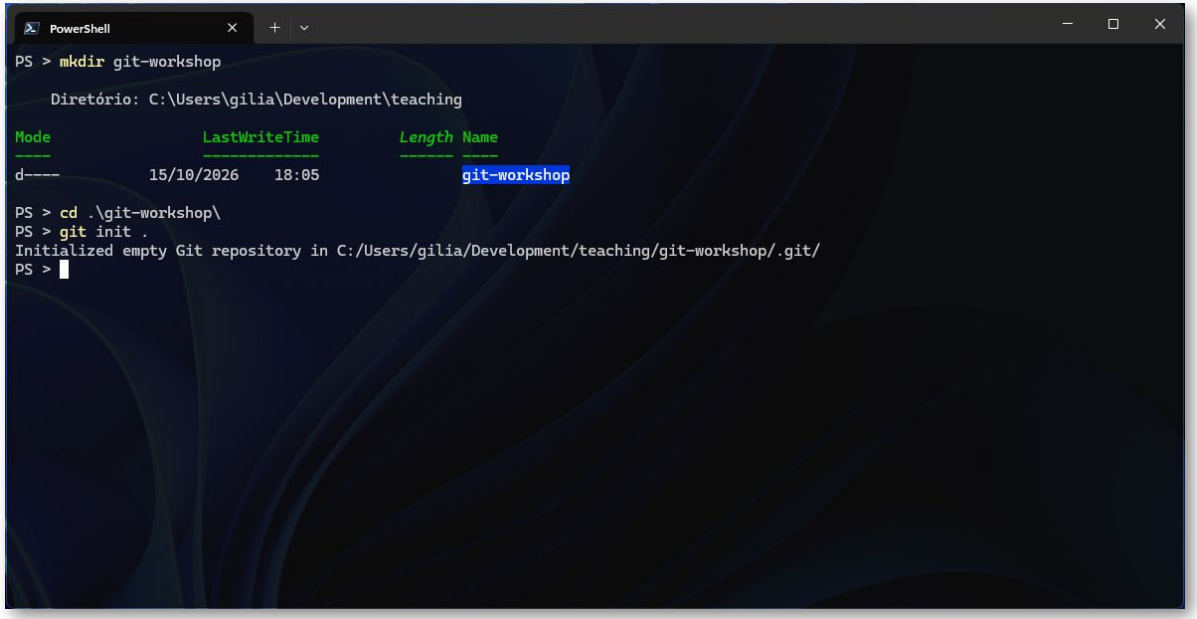
```
git init .

git init projeto-app
```

Inicializando o repositório

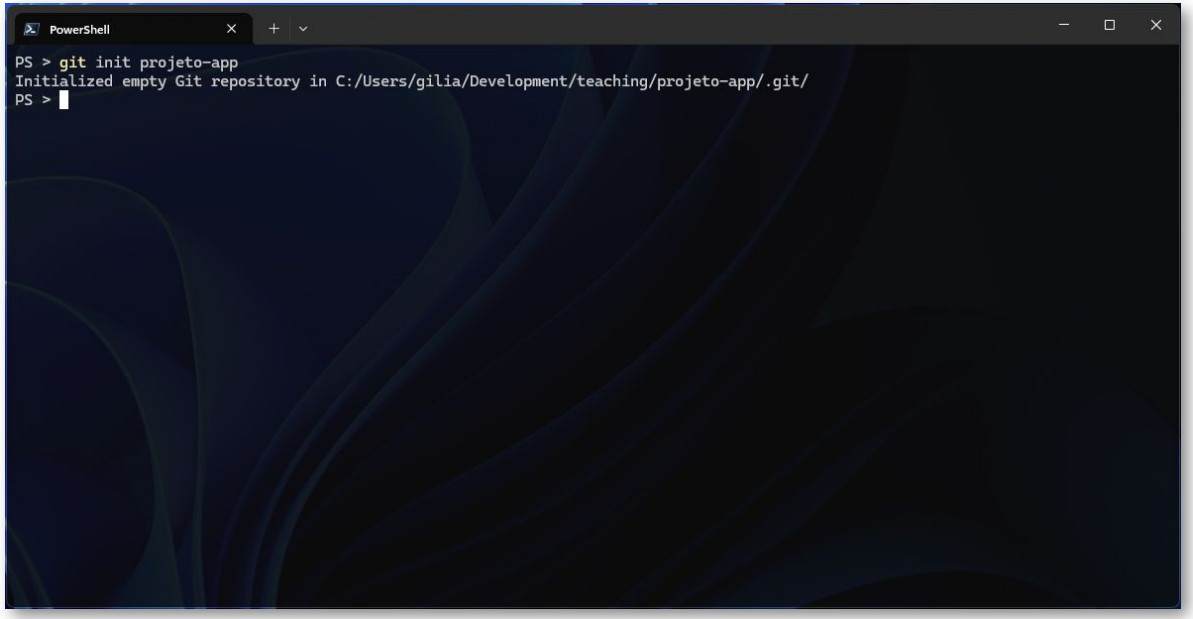
```
mkdir projeto-app
cd projeto-app
git init .
```

Inicializando o repositório “de dentro”



Inicializando o repositório

git init projeto-app



```
$ git init projeto-app

Initialized empty Git repository in
C:/Users/giliard/Development/teaching/projeto-app/.git/

$ cd projeto-app
$ ls
```

A pasta .git

O comando **git init** cria um diretório oculto chamado **.git**, responsável por armazenar todos os dados e metadados necessários ao controle de versão.

Se quisermos parar de usar o Git como ferramenta de versionamento de código basta apagarmos essa pasta do diretório do nosso projeto.

```
projeto-app/
│
├── .git/
│   ├── HEAD
│   ├── config
│   ├── description
│   ├── hooks/
│   ├── info/
│   ├── objects/
│   └── refs/
│
└── ... arquivos do projeto
```

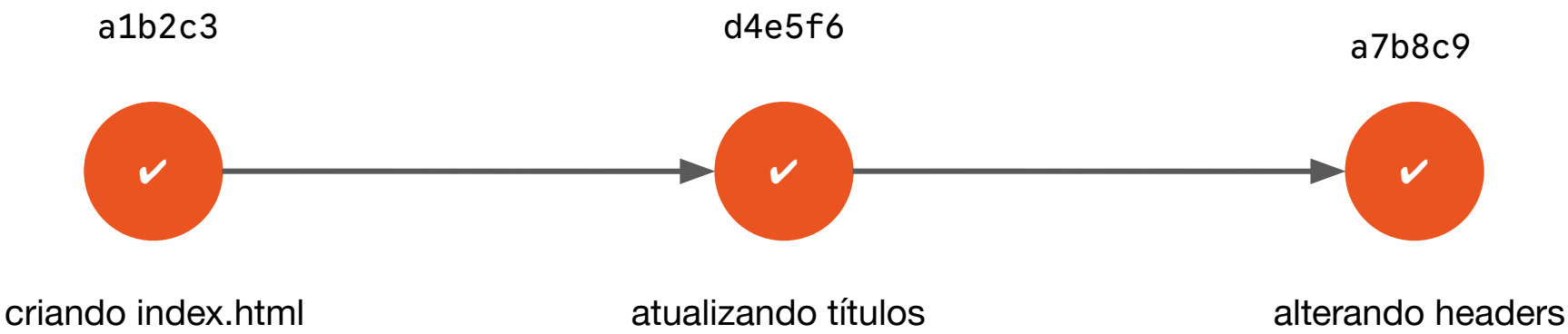
Rastreando alterações do código

O Git permite registrar e acompanhar a evolução de um projeto ao longo do tempo criando um histórico de modificações.

Esse histórico é construído por meio de **commits:**

- ➔ Um **commit é uma fotografia** que registra o estado de um projeto em determinado momento
- ➔ Cada commit possui um identificador (*hash*), uma mensagem descritiva e informações de autoria
- ➔ Os commits estão relacionados entre si, formando um histórico de versões do projeto.

Histórico de alterações



Mas como construímos esse histórico de versões?



A história dos três porquinhos

As três áreas fundamentais do Git

1. **Working directory (diretório de trabalho)**

   É a área onde o desenvolvedor cria, modifica e exclui os arquivos do projeto. Representa o estado atual dos arquivos disponíveis para edição.

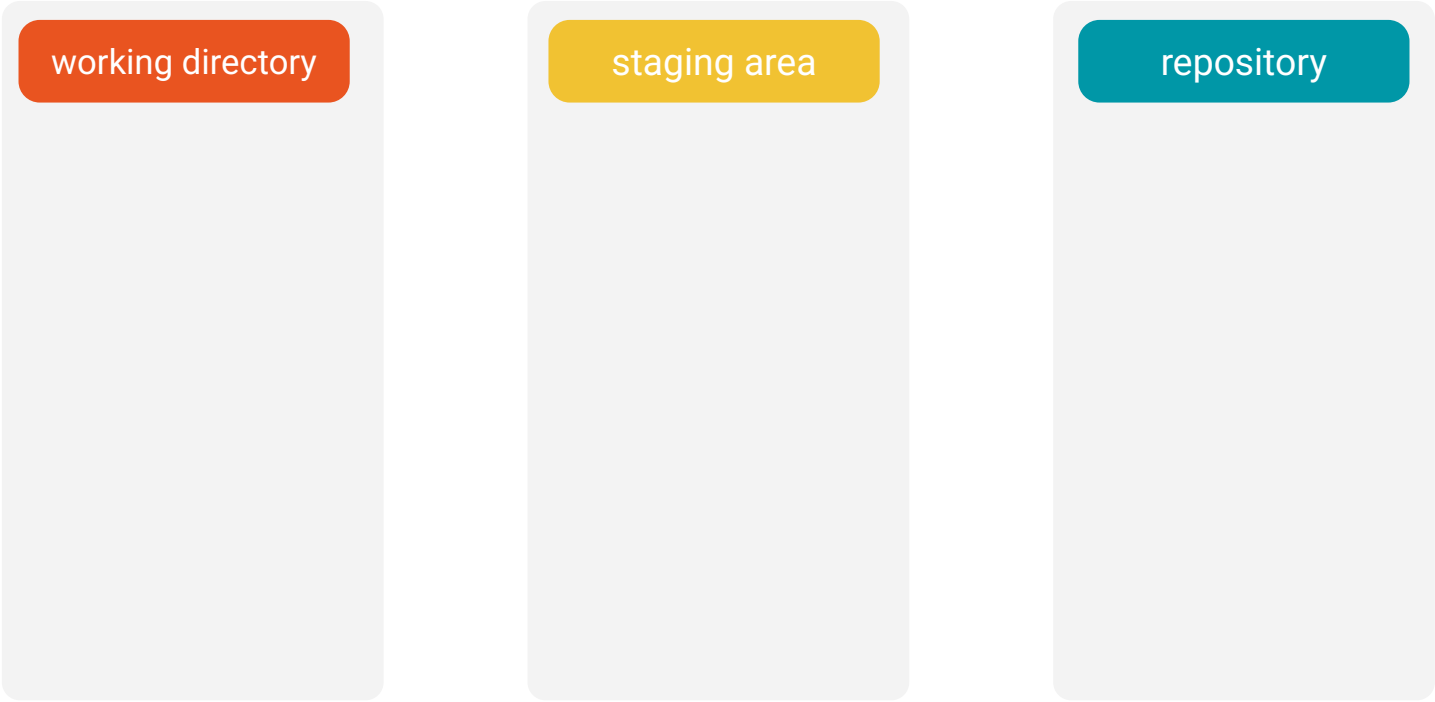
2. **Staging Area (área de preparação)**

   É a área intermediária onde o desenvolvedor seleciona as alterações que deseja incluir no próximo *commit*.

3. **Repository (repositório local)**

   É a área onde o Git armazena permanentemente os *commits* e mantém o histórico de versões do projeto.

As três áreas fundamentais do Git



Verificando o estado do repositório

Ao criar o nosso primeiro arquivo é possível visualizar a **mudança no estado do repositório** com o seguinte comando:

```
git status
```

Todo arquivo é criado no ***working directory*** (diretório de trabalho) que é o espaço disponível ao desenvolvedor para criar, modificar e excluir os arquivos do seu projeto.

```
$ git status

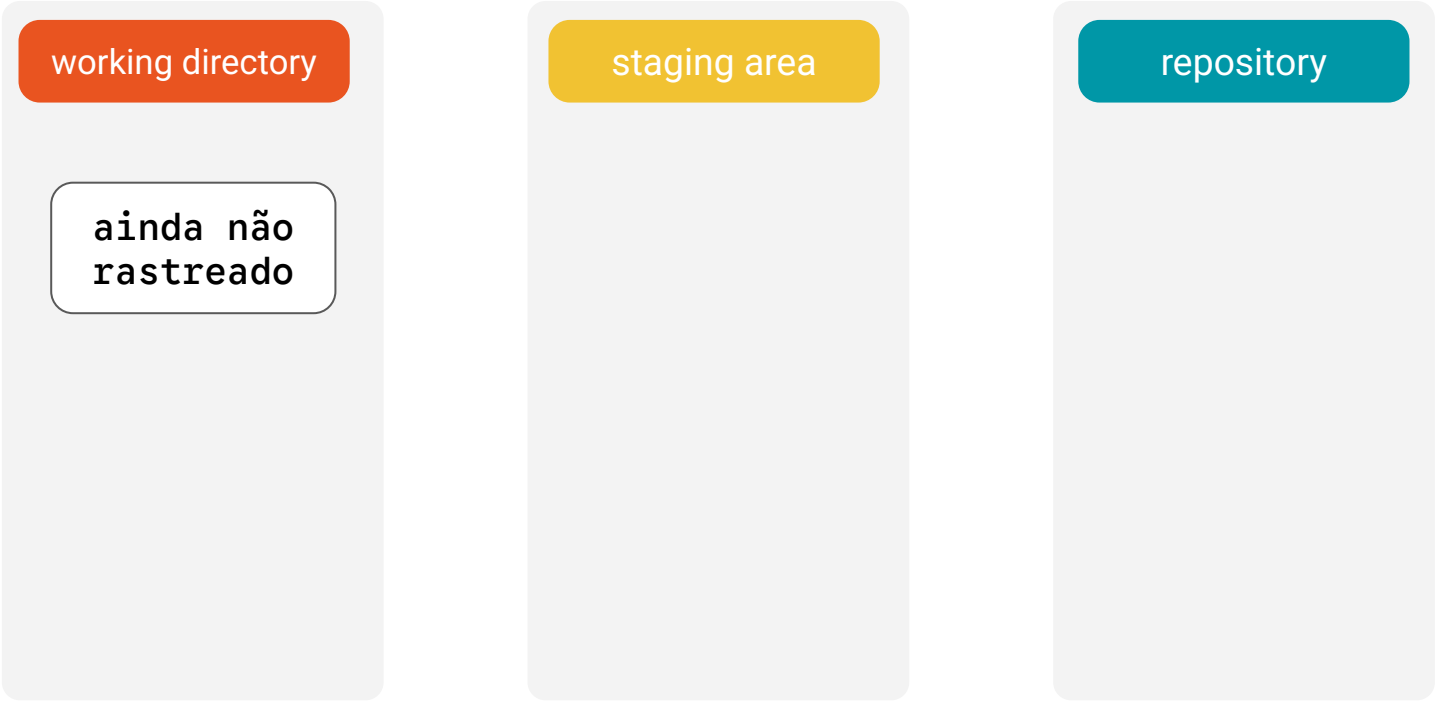
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        exemplo.py

nothing added to commit but untracked files present (use "git
add" to track)
```

Rastreando arquivos arquivos locais



Rastreando arquivos locais

Aos rastrear um arquivo nós estamos informando ao Git quais modificações vão fazer parte de um commit.

Esta é a **área de preparação** (*staged*) para um futuro commit.

Podemos adicionar um arquivo na área de staged com o comando:

```
git add exemplo.py
```

Rastreando arquivos locais

Aos rastrear um arquivo nós estamos informando ao Git quais modificações vão fazer parte de um commit.

Esta é a **área de preparação** (*staged*) para um futuro commit.

Podemos adicionar um arquivo na área de staged com o comando:

```
git add exemplo.py

git add *.py *.txt

git add .
```

Aqui podemos passar:
- ➔ o nome de um ou vários arquivos
- ➔ indicar as extensões de arquivos como, por exemplo, *.html ou *.py
- ➔ ou adicionar todos os arquivos de uma vez com o operador .

Rastreando arquivos

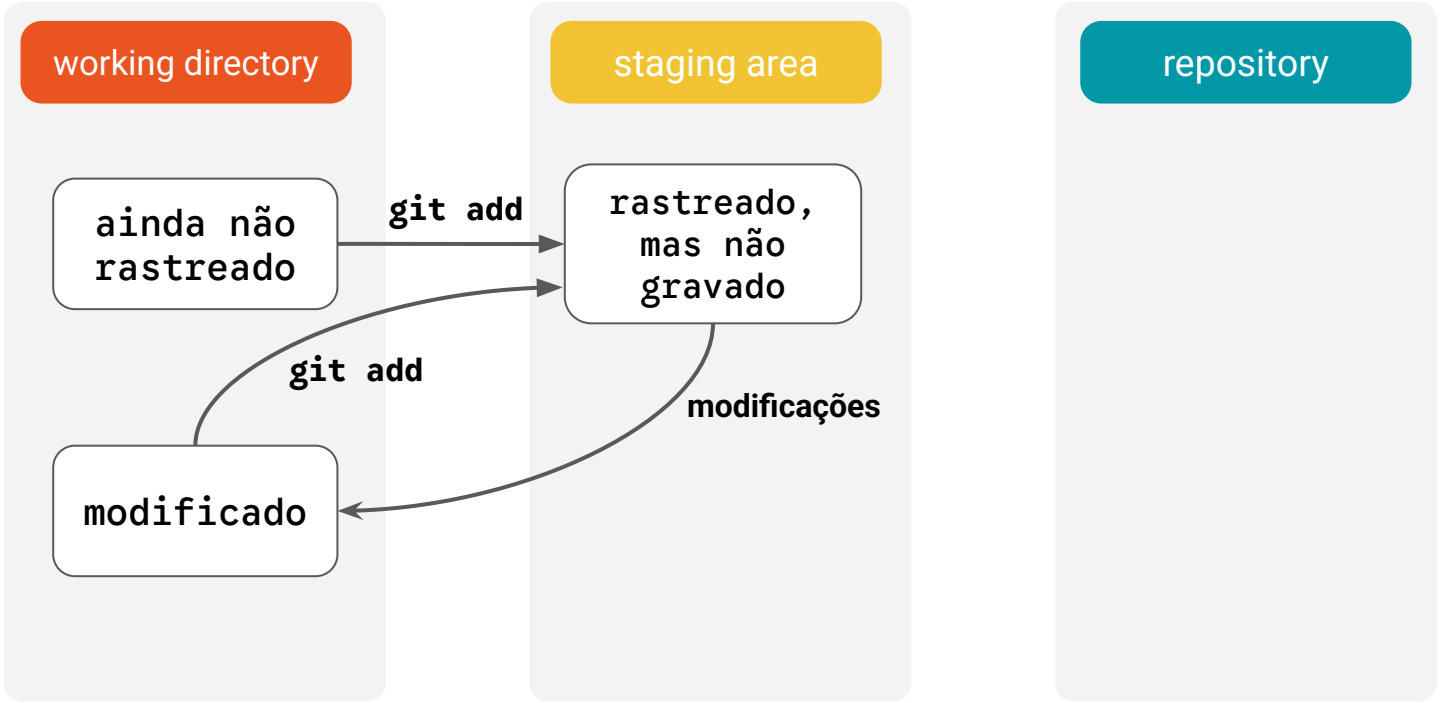


Rastreando arquivos locais

Após um arquivo ser rastreado, o Git identifica as modificações realizadas nele no *Working Directory*.

Para incluir essas alterações na área de *staging*, é necessário adicioná-las novamente com o comando **`git add`**

Rastreando arquivos



```
$ git add .
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   exemplo.py
```

```
$ git add .
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   exemplo.py
```

Essa linha **identifica cada arquivo** adicionado na área de staging e ainda se se trata de um arquivo **novo, modificado, renomeado ou deletado**

Fazendo o meu primeiro commit

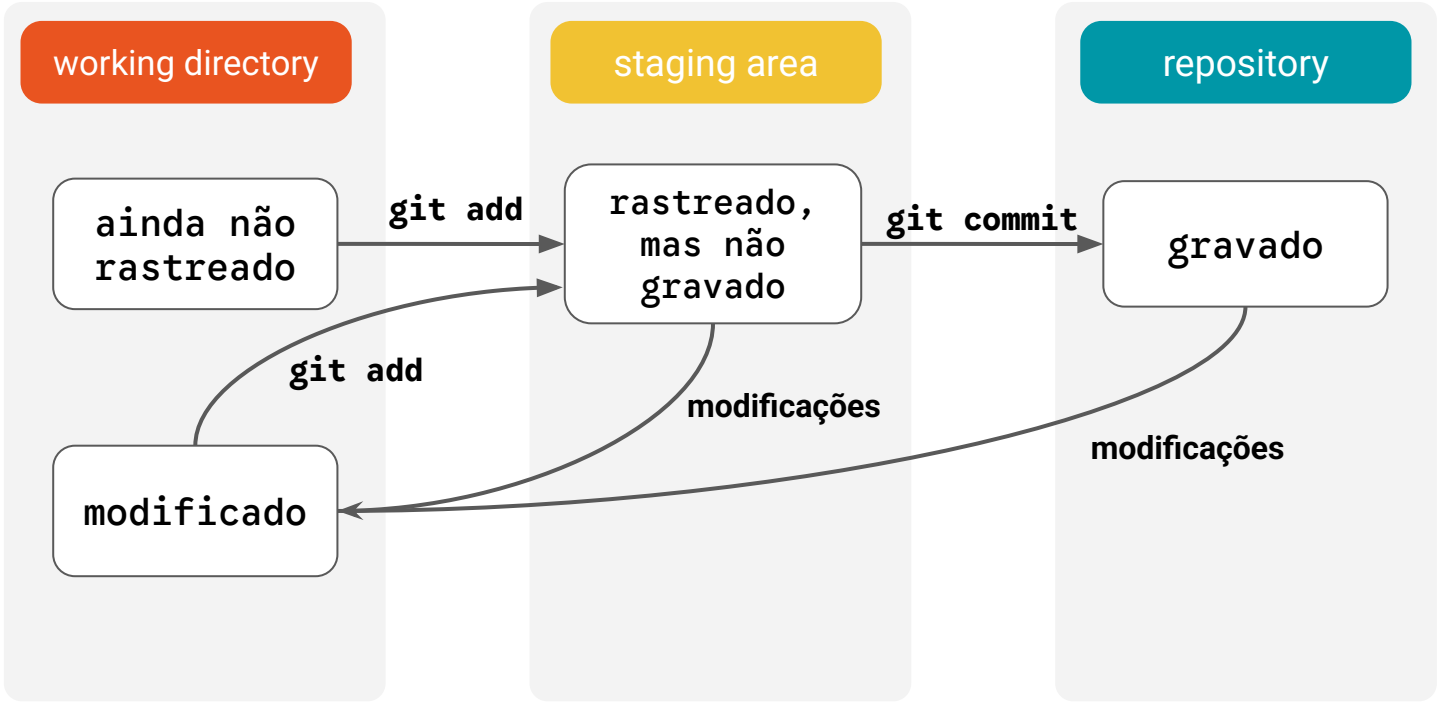
Após preparar todas as alterações que queremos salvar em um commit podemos efetivamente escrever esse commit no histórico de versões do nosso projeto.

Para fazer isso, utilizamos o comando:

```
git commit -m “Meu primeiro commit”
```

A opção **-m** indica que vamos informar a mensagem do commit na linha de comando.

Escrevendo um commit



```
$ git commit -m "Meu primeiro commit"
[main (root-commit) 6c4f4b7] Meu primeiro commit
 1 file changed, 2 insertions(+)
 create mode 100644 exemplo.py
```

Os 4 elementos básicos de um commit

1. Possui um **identificador hash**
2. Um conjunto de **alterações** em um ou mais arquivos
3. Uma **mensagem** descritiva
4. Um usuário **autor** do commit

Lembre-se, a nossa primeira tarefa foi configurar o nome e email do usuário do Git.

Relembrando...

Os comandos que vimos até agora foram:

```
git init .

git status

git add exemplo.py

git commit -m “mensagem descritiva”
```

Pergunta...

O que acontece quando alteramos novamente o arquivo que já foi comitado?

Pergunta...

O que acontece se digitarmos somente o comando **`git commit`** sem o parâmetro **`-m`** e a mensagem?

Gerenciando arquivos (comandos frequentes)

O Git oferece comandos que **simplificam operações frequentes** para o gerenciamento de arquivos como **renomear, mover ou excluir arquivos**:

1. **`git mv`** → move ou renomeia um arquivo e prepara a alteração
2. **`git rm`** → remove um arquivo e prepara sua exclusão

Essas alterações são automaticamente enviadas para a área de *staging*

```
$ git mv .\exemplo.py main.py
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    exemplo.py -> main.py

$ git commit -m “alterando nome do arquivo”
[main fbbcd48] alterando nome do arquivo
 1 file changed, 0 insertions(+), 0 deletions(-)
 rename exemplo.py => main.py (100%)
```

Remover um arquivo é um pouco mais complicado…

Quando falamos em remover um arquivo, temos que pensar de onde queremos removê-lo e o que fazer com o seu histórico de modificações.

Comando	Working Directory	Staging Area
`git rm arquivo.py`	Exclui o arquivo	Prepara sua exclusão
`git rm --cached arquivo.py`	Mantém o arquivo	Remove o arquivo do índice, preparando sua exclusão se já estava versionado
`git rm -f arquivo.py`	Exclui o arquivo, mesmo com alterações locais	Prepara sua exclusão
`git restore arquivo.py`	Descarta alterações não preparadas, restaurando o conteúdo a partir da Staging Area	Não modifica
`git restore --staged arquivo.py`	Mantém o arquivo	Restaura o estado do último commit (HEAD)

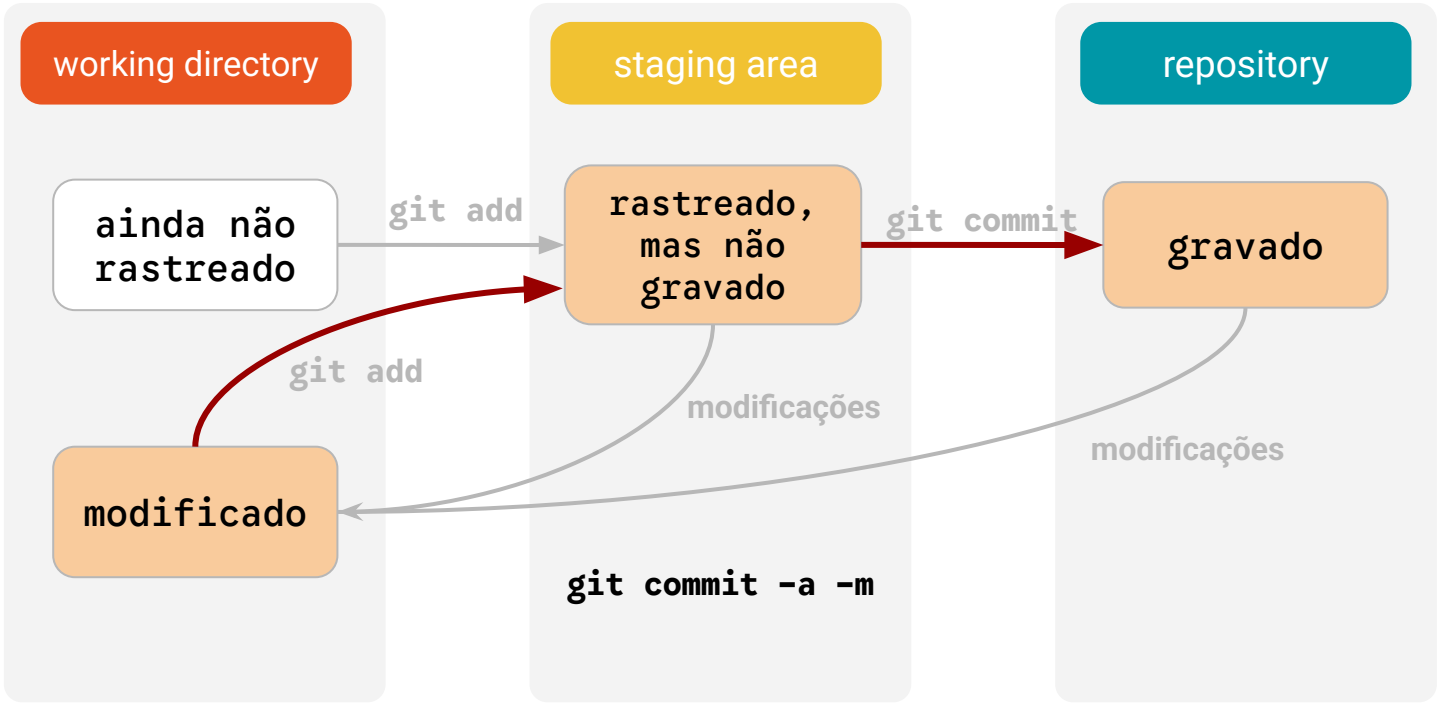
Rastreando e comitando mudanças de uma só vez

Uma vez que um arquivo foi mandado para a área de *staged*, é possível rastrear e comitar as próximas mudanças de uma só vez adicionando a opção **`-a`** **ao comando `git commit`**

```
git commit -am “novas alterações”
```

```
git commit -a -m “novas alterações”
```

Rastreando e comitando mudanças de uma só vez



Exibindo o histórico de commits

O Git permite listar o histórico de commits no próprio terminal com o seguinte comando:

```
git log
```

Quais as informações que podemos identificar no histórico de commits?

```
$ git log
commit 6c4f4b7c3d9be5f604e26f15794acf96b63c4f05 (HEAD -> main)
Author: Giliard Godoi <giliard.godoi@fatecourinhos.edu.br>
Date:   Thu Oct 15 16:32:18 2026 -0300

    Meu primeiro commit
```

Exibindo o histórico de commits

A medida que o histórico de commits cresce, é comum exibir apenas um resumo dessas informações.

Para tanto existem diversas opções e parâmetros para configurar essa exibição.

```
git log --oneline -n5
```

Onde:

- ➔ **`--oneline`**: exibe cada commit em uma única linha
- ➔ **`-n5`**: exibe os **n** primeiros commits apenas

```
$ git log --oneline -n10 --graph

* fbbcd48 (HEAD -> main) alterando nome do arquivo
* 6c4f4b7 Meu primeiro commit
```

Ignorando arquivos com .gitignore

Em um projeto de desenvolvimento de softwares é comum gerar arquivos intermediários como, por exemplos, arquivos de buildings, *caches* de compilação etc.

Além disso, podemos ter pastas como a **`.vscode`** que armazena configurações locais do VSCode.

O arquivo **`.gitignore`** define arquivos e subdiretórios inteiros para serem ignorados pelo Git.

github/gitignore: A collection of useful .gitignore templates

```
# ignorando arquivos específicos
settings.ini
secrets.ini
db.sqlite3
build

# ignorando diretórios inteiros
build/
dist/

# ignorando extensões de arquivos
*.png
*.jpeg
*.log
```

Mantendo diretórios vazios

Às vezes pode ser interessante manter uma estrutura de subdiretórios no nosso projeto, contudo, por padrão, o Git somente monitora arquivos.

Diretórios vazios não são percebidos pelo Git como mudanças no estado do repositório. Em outras palavras não é possível fazer **`git add`** ou **`git commit`** em um repositório vazio.

Uma saída é criar um arquivo vazio nesse diretório e nomeá-lo como **`.gitkeep`**

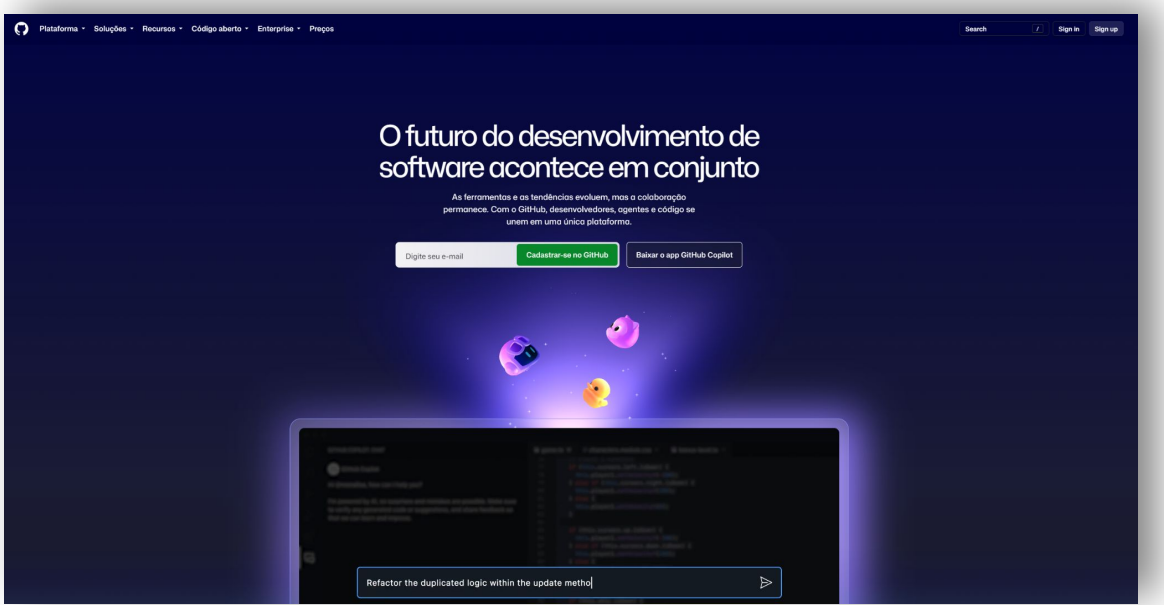
Rastreando e comitando o arquivo **`.gitkeep`** é um padrão para indicar que aquele diretório não possui arquivos importantes para o projeto, o que queremos manter é o nome do diretório.

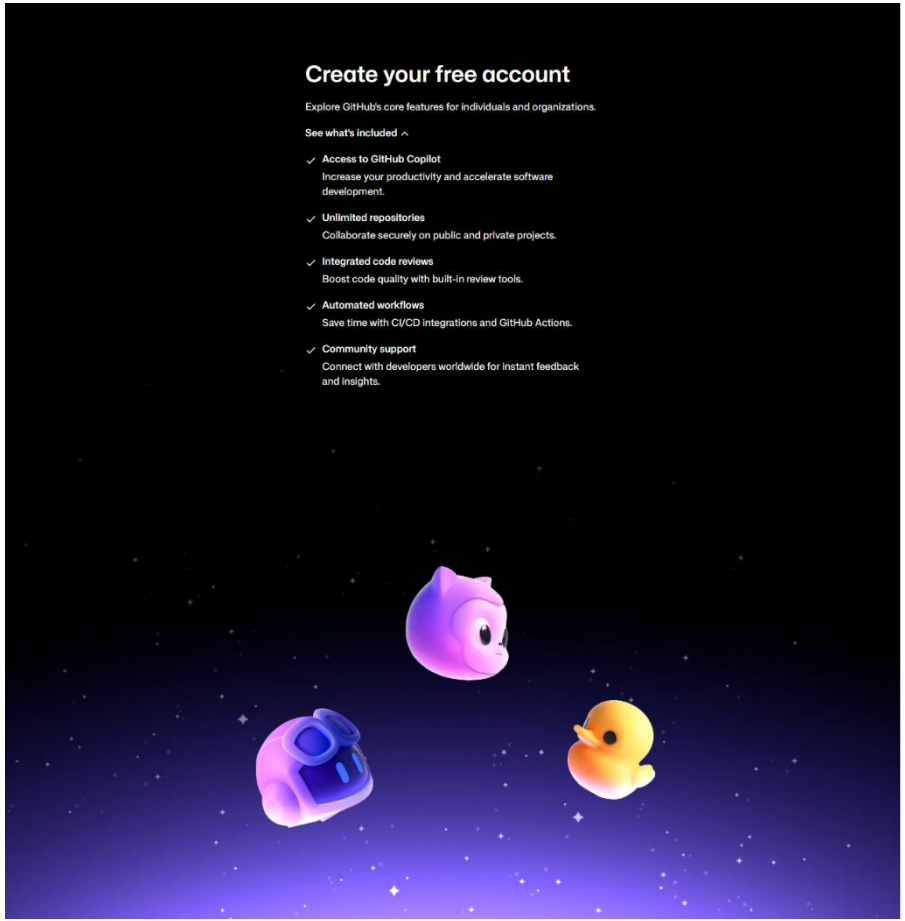


Repositório remotos no GitHub

O que é o GitHub?

O GitHub é uma plataforma para hospedagem de repositórios remotos de versionamento de códigos. Disponível <https://github.com/>





Already have an account? Sign in →

Sign up for GitHub

Continue with Google

Continue with Apple

or

Email*

Email

Password*

Password

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username*

Username

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

Your Country/Region*

Brazil

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Email preferences

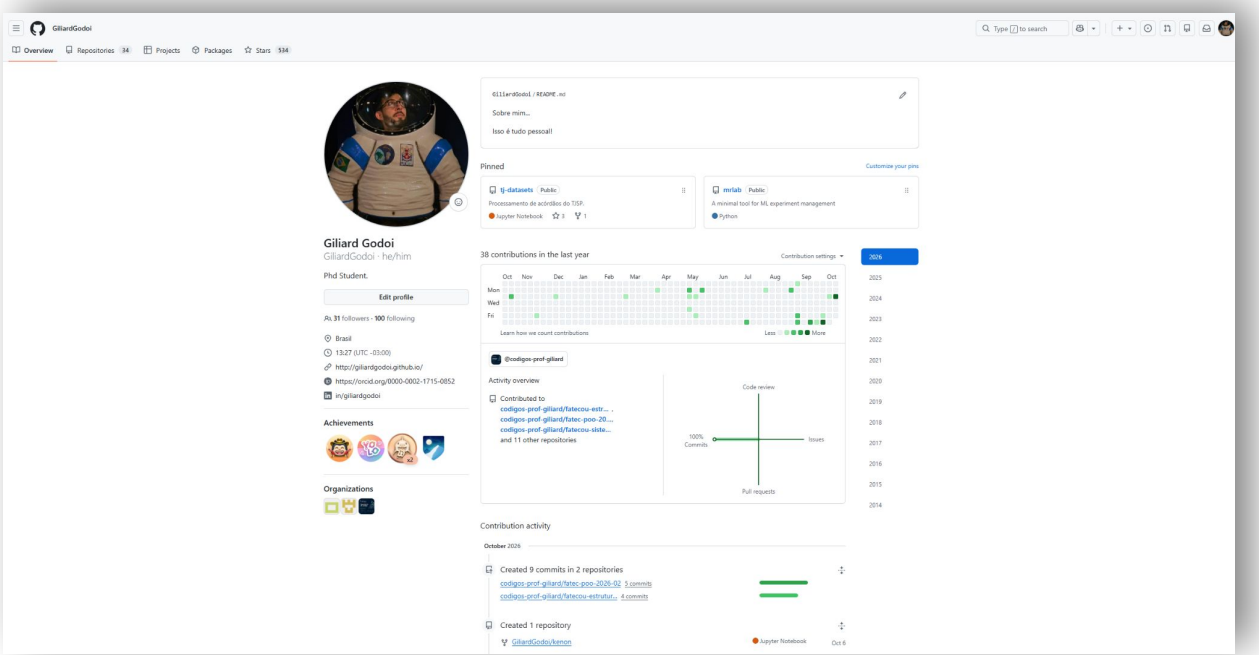
Receive occasional product updates and announcements

Create account ›

By creating an account, you agree to the Terms of Service. For more information about GitHub's privacy practices, see the GitHub Privacy Statement. We'll occasionally send you account-related emails.

O que é o GitHub?

Ou é uma rede social para desenvolvedores e de projetos **open-source**.



Alternativas

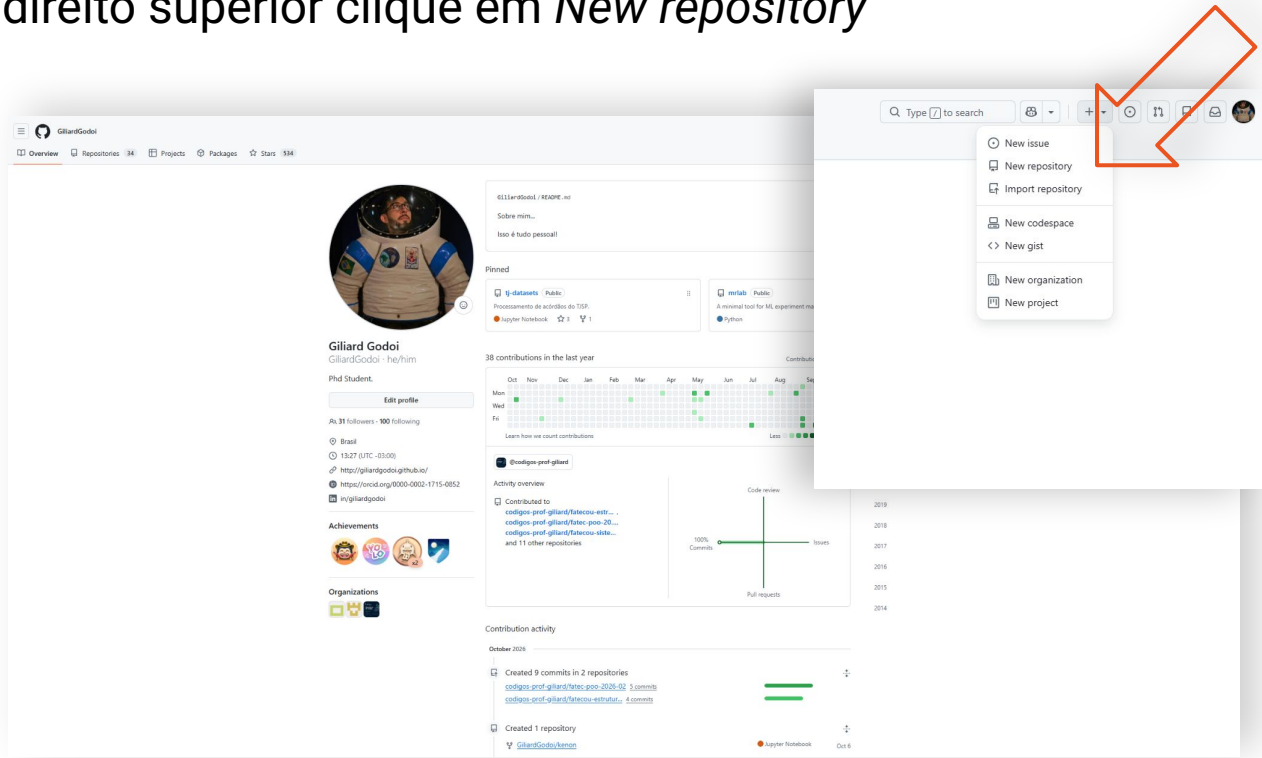
Apesar da predominância do GitHub, existem outras plataformas semelhantes:

1. **GitLab** <https://about.gitlab.com/pt-br/>
2. **SourceForge** <https://sourceforge.net/>
3. **BitBucket** <https://bitbucket.org/>

Saiba mais em <Best GitHub Alternatives for Developers in 2026 - DEV Community>

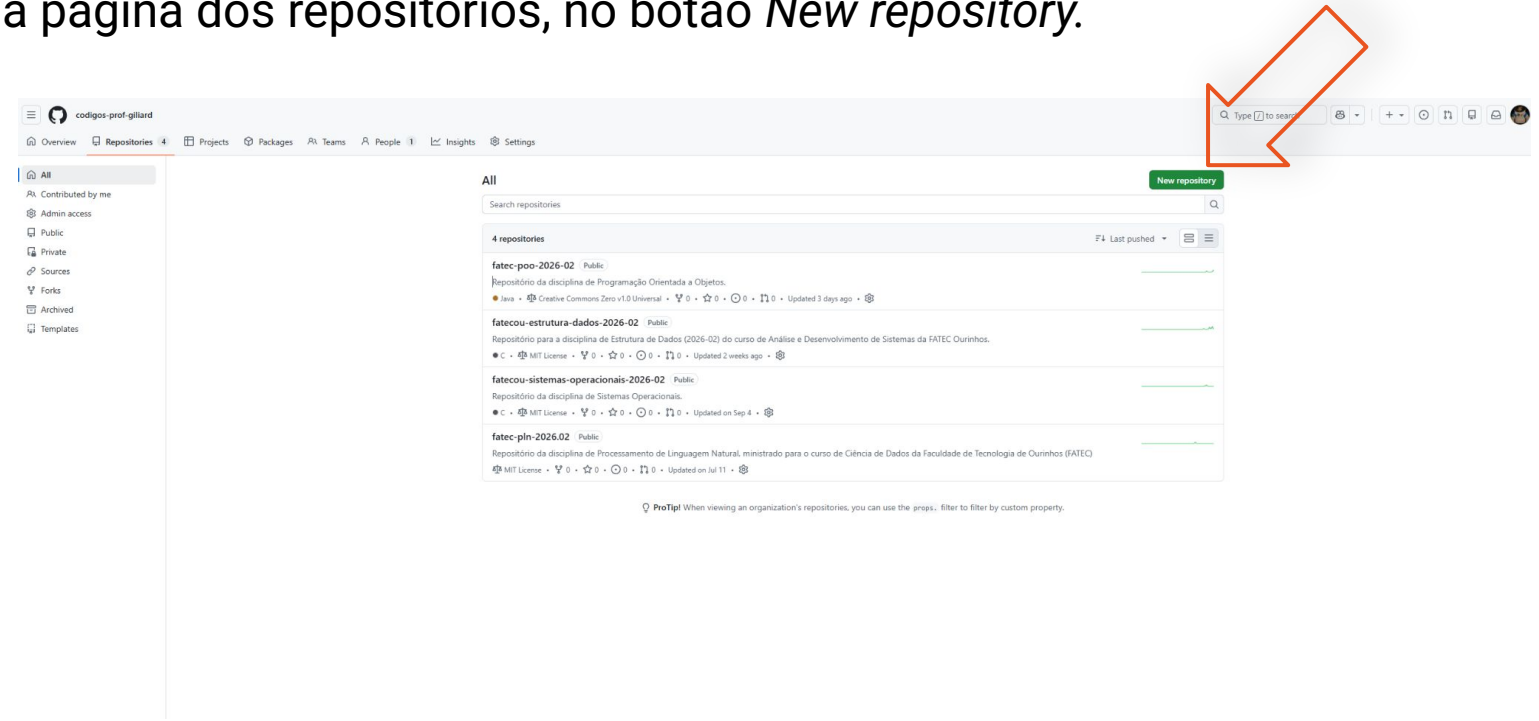
Criando um novo repositório remoto

No canto direito superior clique em *New repository*

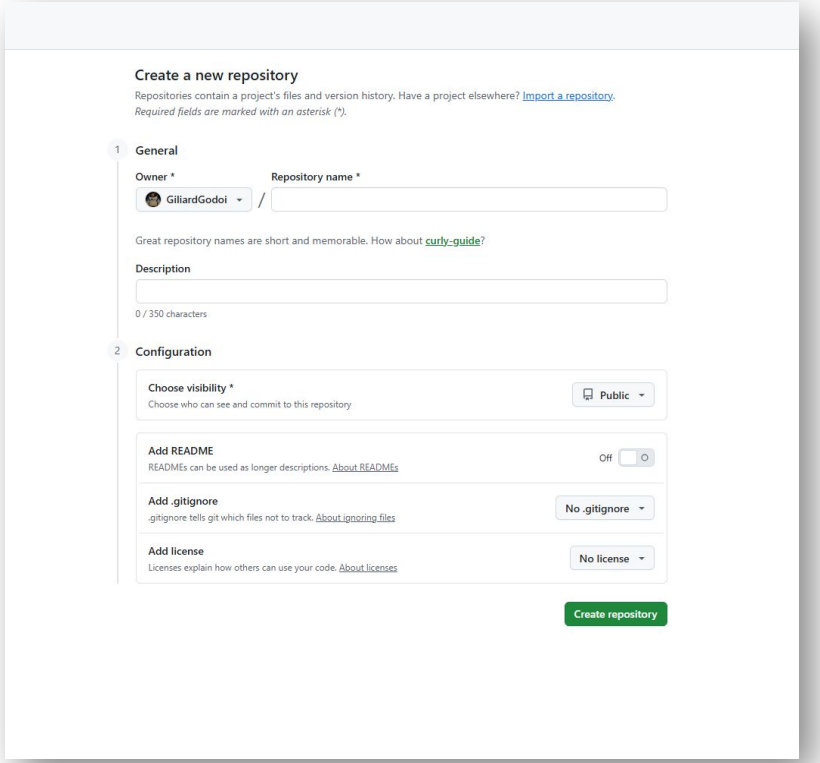


Criando um novo repositório remoto

Ou na página dos repositórios, no botão *New repository*.



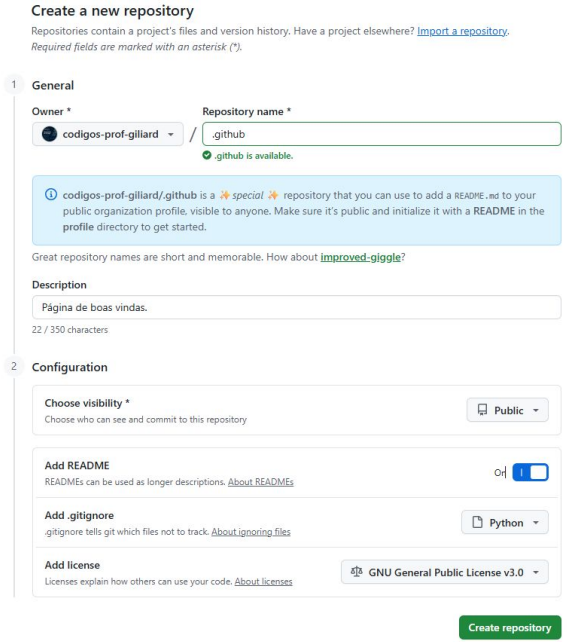
Criando um novo repositório remoto



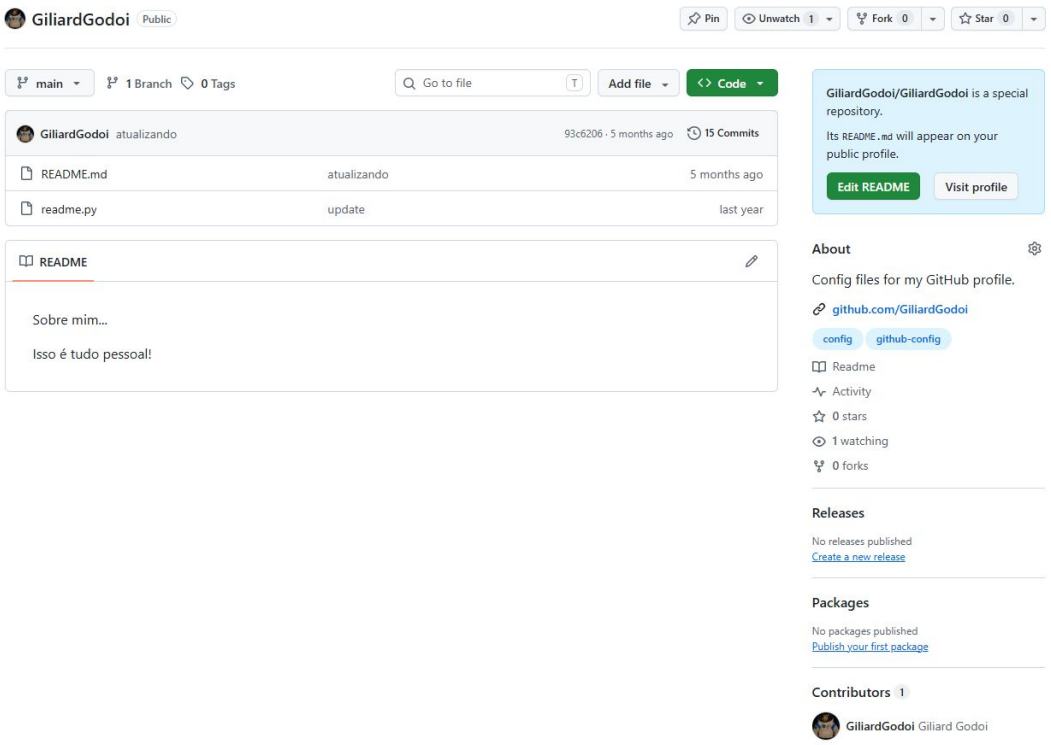
Meu primeiro repositório

O GitHub permite criar repositório com capacidades especiais

1. **Repositório com o mesmo nome do usuário**. Ou repositório **.github.** Altera a exibição de uma mensagem na página inicial do seu perfil.
2. Repositório com o mesmo nome do usuário seguido de .**github.io** Permite criar uma página web estática para você.

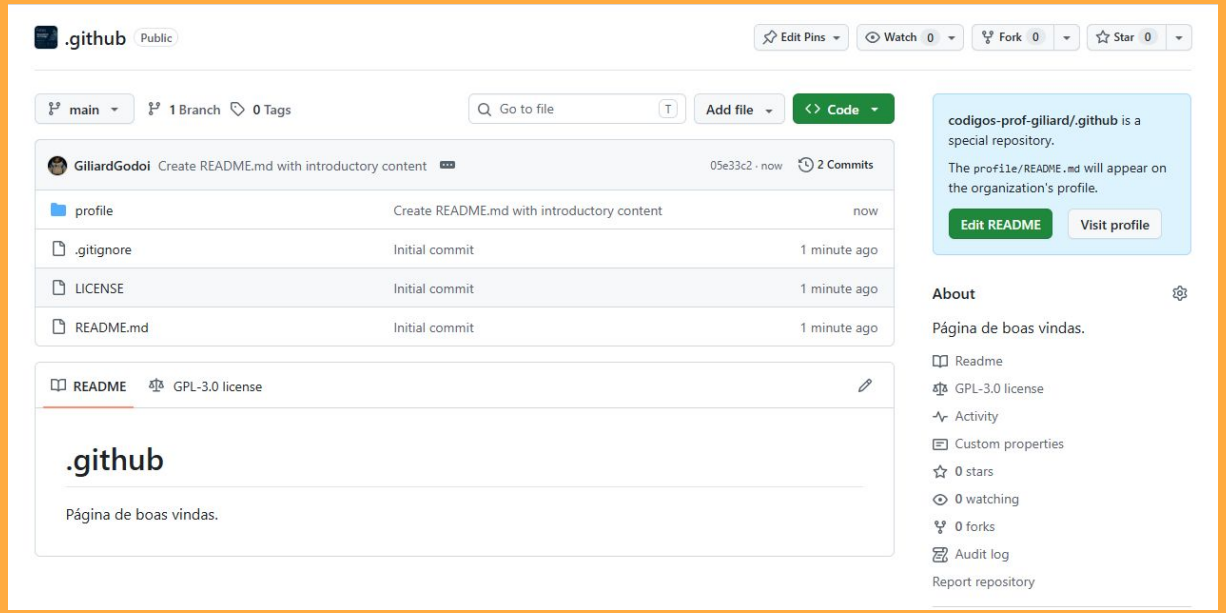


Mensagem de Boas-vindas em Perfil Pessoal



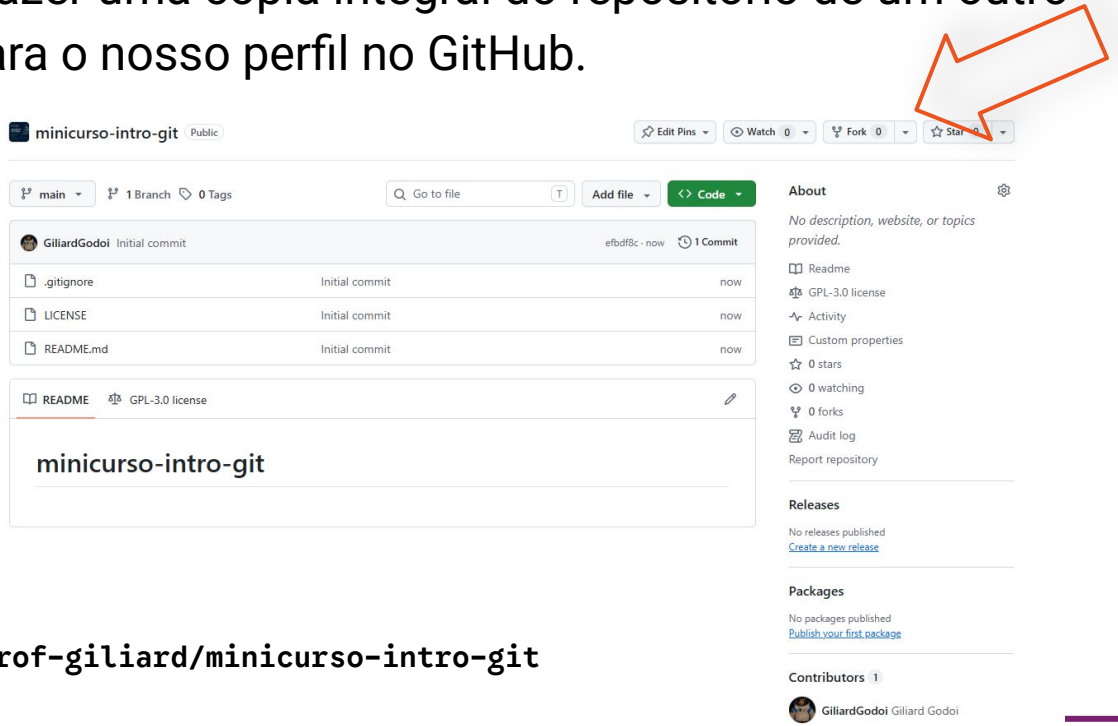
Mensagem de Boas-vindas em Organizações

Nesse caso nós precisamos criar o arquivo em **.github/profile/README.md**



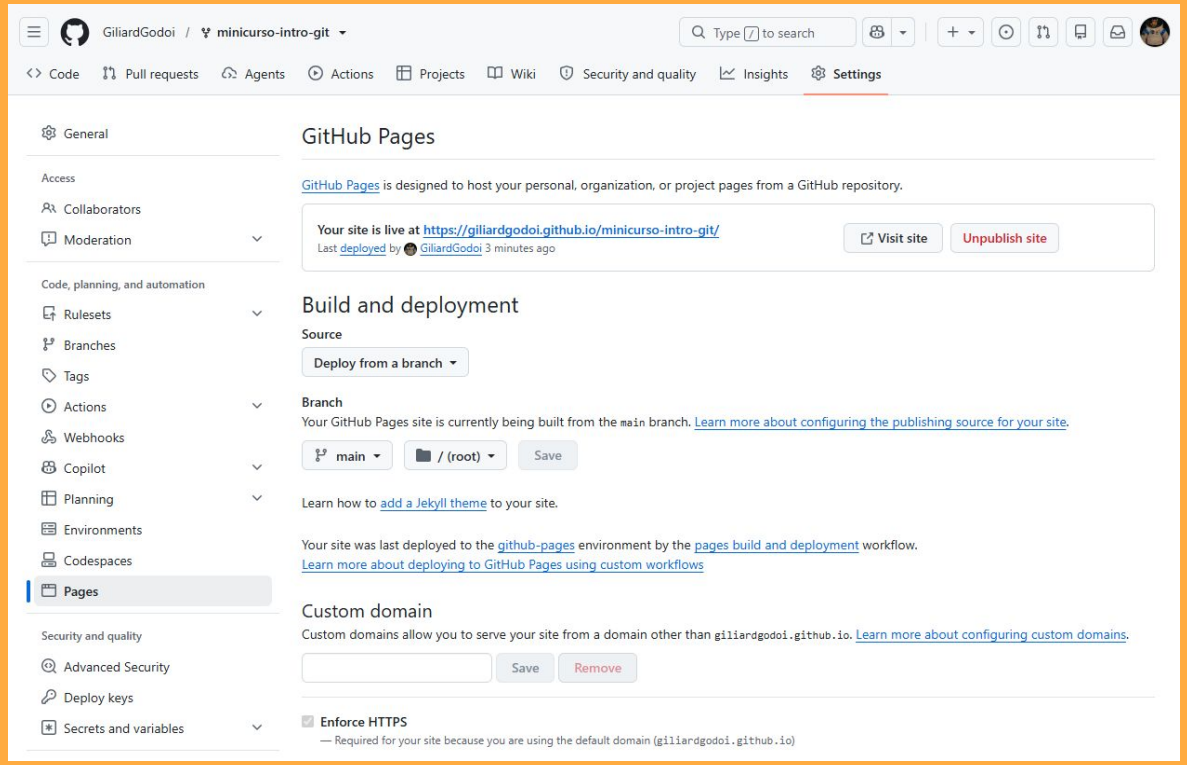
Fork de um repositório

Podemos ainda fazer uma cópia integral do repositório de um outro desenvolvedor para o nosso perfil no GitHub.



github.com/codigos-prof-giliard/minicurso-intro-git

Configurar GitHub Pages



Clonar um repositório

Clonar um repositório é **baixar um repositório remoto** (hospedado no GitHub, por exemplo) **para a sua máquina local**:

```
git clone <url Repositório> <nome diretório [opcional]>
```

```
git clone https://github.com/SeuUserName/minicurso-intro-git.git
```

```
$ git clone https://github.com/GiliardGodoi/minicurso-intro-git.git

Cloning into 'minicurso-intro-git'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.
```

Sincronizar um repositório local com um remoto

1. Todo repositório local possui uma referência para um endereço de um repositório remoto.
2. Dessa forma podemos enviar nossas alterações locais para o remoto.
3. A referência principal é chamada de **origin**
4. Inclusive podemos **listar todas as referências de repositórios remotos** com o seguinte código

```
git remote -v
```

```
$ cd GiliardGodoi
$ git remote -v

origin  https://github.com/GiliardGodoi/GiliardGodoi.git (fetch)
origin  https://github.com/GiliardGodoi/GiliardGodoi.git (push)
```

Sincronizar um repositório local com um remoto

Digamos que você começou a desenvolver o seu projeto em um repositório local, e agora deseja publicá-lo em um repositório remoto do GitHub.

Nós **adicionar o endereço do repositório remoto** como uma referência no repositório local e empurrar os históricos de versões para o repositório remoto.

Para tanto, usamos o seguinte comando:

```
git remote add origin <url Repositório>
```

```
$ cd projeto-app
$ git remote -v

$ git remote add origin https://github.com/GiliardGodoi/GiliardGodoi.git
$ git remote -v

origin  https://github.com/GiliardGodoi/GiliardGodoi.git (fetch)
origin  https://github.com/GiliardGodoi/GiliardGodoi.git (push)
```

Subir o histórico de revisões

Para enviar as alterações locais para o repositório remoto, utilizamos o seguinte comando:

```
git push
```

No primeiro envio talvez seja necessário indicar exatamente qual o repositório remoto e qual a sua branch que queremos sincronizar.

```
git push --set-upstream origin main
```

Baixar o histórico de revisões

Para baixar quaisquer alterações do repositório remoto para o nosso repositório local, temos que executar o comando:

```
git pull
```

```
git pull --set-upstream origin main
```

Baixar o histórico de revisões

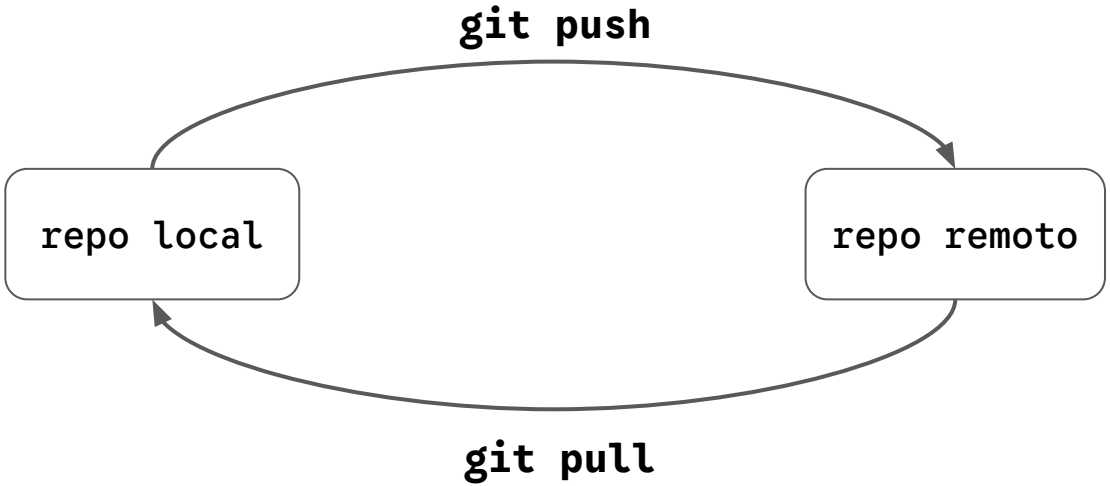
Se porventura fizermos um commit antes de baixar as atualizações do repositório remoto, podemos tentar aplicar os commits locais a partir do HEAD do repositório remoto. Isto é, se não houver nenhum potencial conflito.



```
git pull --rebase
```

Ciclo de trabalho constante

Sempre mantenha os repositórios locais sincronizados para **evitar conflito** de versões de um mesmo arquivo.

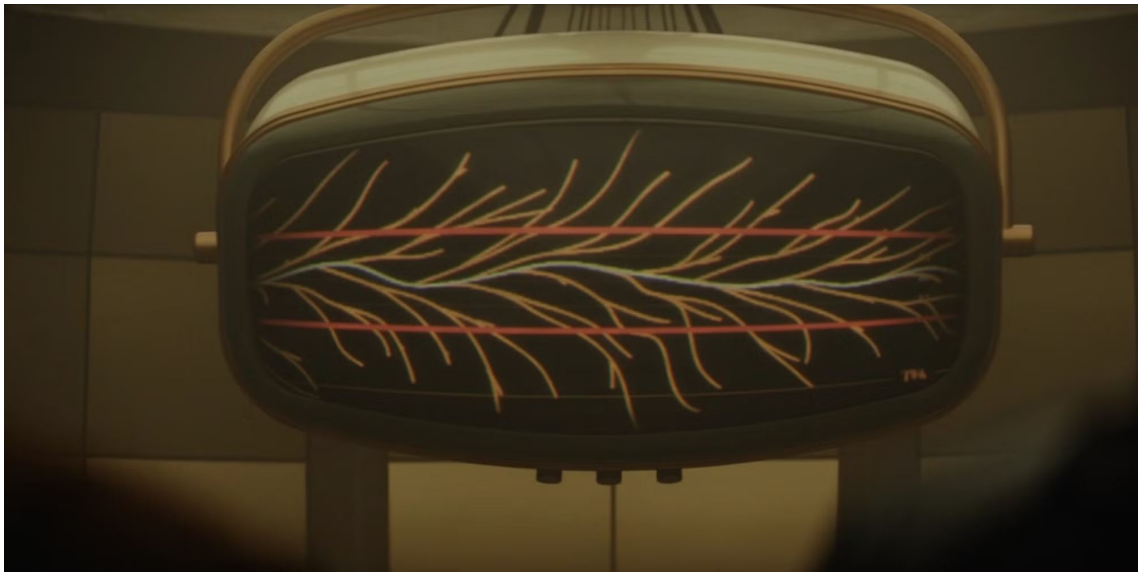




Branches

Fatec Ourinhos

Você já ouviu falar nas linhas do tempo do MCU?



O que é uma branch?

Uma branch (ramificação) permite criar linhas de desenvolvimento independentes.

Isso permite desenvolver funcionalidades, corrigir erros ou experimentar alterações sem alterar a linha principal (main) de desenvolvimento do projeto.

- A branch **`main`** é normalmente utilizada como a linha principal de desenvolvimento
- Novas branches permitem trabalhar em diferentes funcionalidades sem alterar diretamente a **`main`**
- À medida que novos commits são criados, a referência da branch avança automaticamente
- As alterações de diferentes branches podem ser integradas por meio de operações como **`merge`** ou **`rebase`**.

Criar uma nova branch

Criamos uma nova branch com o comando git branch seguido do nome da branch:

```
git branch feature
```

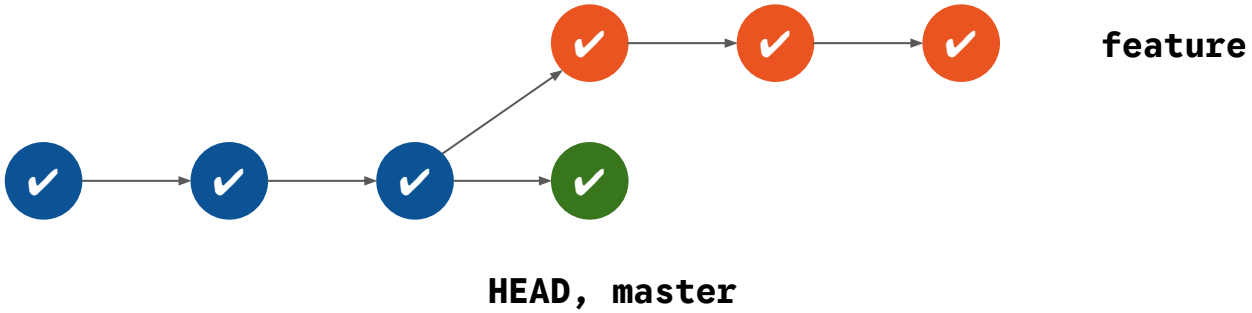
Nomes semânticos para branches

Utilizar nomes semânticos facilita a identificação do objetivo de cada linha de desenvolvimento, tornando o histórico mais organizado e compreensível.

Prefixo	Finalidade	Exemplo
`feature/`	Desenvolvimento de nova funcionalidade	`feature/busca-alunos`
`fix/`	Correção de erros	`fix/calculo-media`
`refactor/`	Refatoração do código	`refactor/classe-aluno`
`docs/`	Atualização da documentação	`docs/manual-usuario`
`test/`	Criação ou manutenção de testes	`test/validacao-cpf`
`chore/`	Tarefas de manutenção	`chore/atualizar-dependencias`

Criar uma nova branch

```
git branch fix-feature
```



Listar as branches locais

Para listar as branches disponíveis no repositório local simplesmente usamos o comando. O parâmetro **`--list`** ou **`-l`** é opcional.

`git branch`

```
$ git branch
  fix-feature
  * main
```

Alternar entre branches

Para alternar entre a branch atual e outra usamos o comando

```
git switch feature
```

```
git switch <nome da branch>
```

Salvar mudanças temporariamente

O Git permite alternar entre branches mesmo quando existem alterações no Working Directory ou na Staging Area, desde que essas alterações possam ser preservadas.

Caso a troca possa sobrescrever modificações locais, o Git impede a operação por segurança.

Nesses casos podemos utilizarmos de memórias temporárias para guardar essas alterações.

`git stash`

Git stash

Git stash é um recurso que permite guardar temporariamente alterações não commitadas no Working Directory e na Staging Area, possibilitando interromper uma tarefa e retomá-la posteriormente.

Comando	Finalidade
`git stash`	Guarda temporariamente alterações em arquivos rastreados
`git stash push -m "mensagem"`	Guarda alterações com uma descrição
`git stash pop`	Reaplica as alterações e remove o stash se a aplicação for bem-sucedida
`git stash apply`	Reaplica as alterações sem remover o stash
`git stash list`	Lista as alterações temporariamente armazenadas
`git stash show -p`	Mostra as diferenças do stash mais recente
`git stash drop`	Exclui um stash armazenado

Criar e alternar entre branches

Também podemos criar e alternar entre branches em um mesmo comando.

Para tanto, basta adicionar o comando **`--create`** ou simplesmente **`-c`**

```
git switch -c bug-fix
```

A nova branch é criada a partir da branch atual. Além disso podemos definir um ponto (***commit***) a partir de onde queremos definir a nova branch.

```
git switch -c bug-fix efbdf8c
```

Enviar uma branch local para repositório remoto

O envio de uma branch local para um repositório remoto pode ser feita em dois contextos.

1. Criar um espelho da branch local também repositório remoto:

```
git push --set-upstream origem new-bug
```

2. Alinhar uma branch existente com uma branch remota que já foi criada:

```
git fetch origin new-bug-reported
git branch --set-upstream-to=origin/new-bug-reported new-bug
```

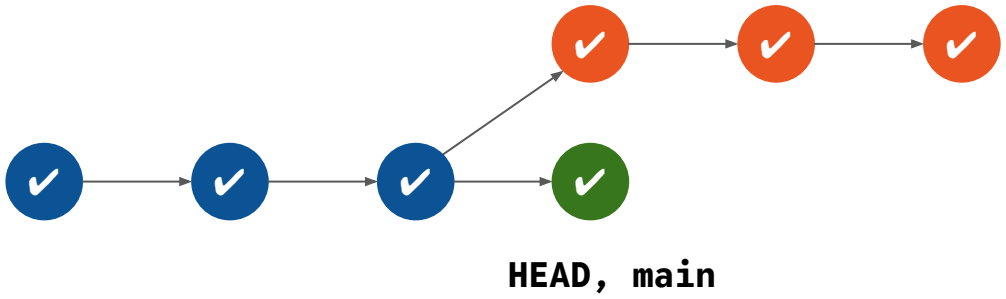
Baixar uma branch de um repositório remoto

Para baixar um branch remoto também podemos fazer:

```
git switch -c new-feature origin\new-feature-dev
```

Git merge

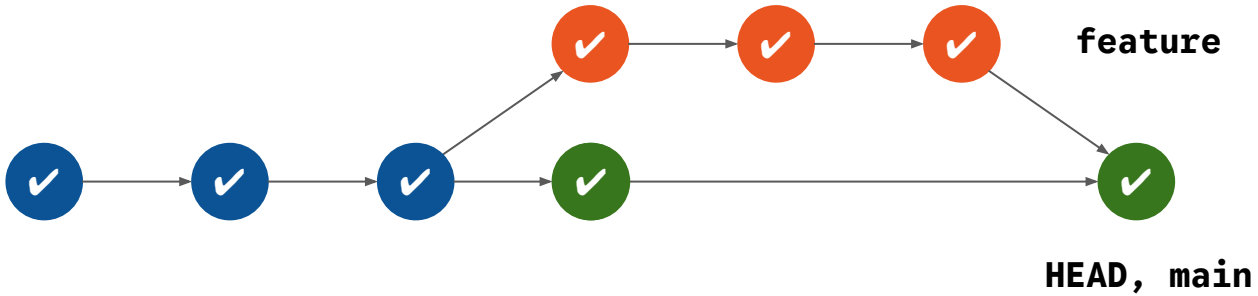
O **merge** consiste na tarefa de juntar duas linhas de evolução do seu código.



Para realizar o **merge**, é necessário estar no branch que **deverá receber** as alterações.

Git merge

Essa junção é sinalizada criando-se um novo commit no branch em que vai ser aplicado o merge.




```
git merge new-feature
```

Conflitos no merge

Um conflito ocorre quando o Git não consegue integrar automaticamente as alterações realizadas em diferentes branches, sendo necessária a intervenção do desenvolvedor para decidir como combiná-las.

Um conflito pode ocorrer por:

1. Alterações na mesma região (linhas) de um arquivo (mais comum)
2. Arquivos com o mesmo nome
3. Uma branch exclui um arquivo enquanto outra modifica.
4. Duas branches renomeia um mesmo arquivo para nomes diferentes.

Resolvendo conflitos

Conflitos de conteúdo são identificados por marcadores especiais:

```
<<<<<<< HEAD
def soma(valores):
=======
def soma_valores_positivos(valores):
>>>>>>> bug-fix
```

Acredite, ninguém gosta de resolver conflitos!!

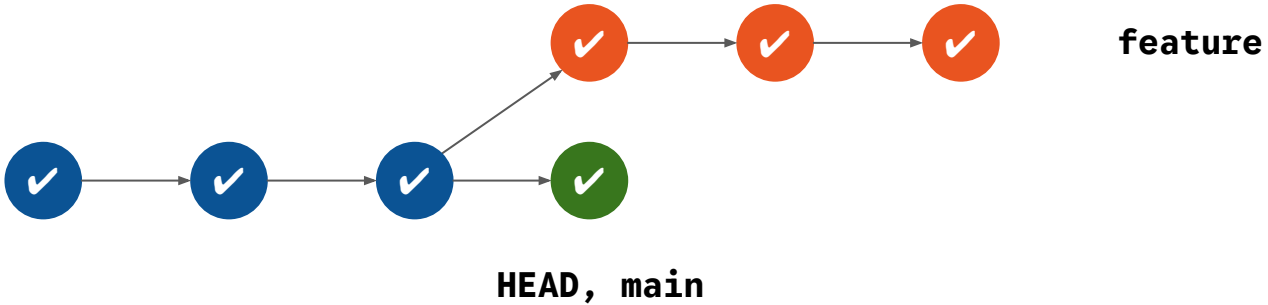
Resolvendo conflitos

O git oferece algumas funcionalidades para resolver conflitos.

Comando	Finalidade
`git diff`	Examina as diferenças e regiões conflitantes.
`git checkout --ours arquivo.py`	Seleciona a versão da branch atual para o arquivo inteiro.
`git checkout --theirs arquivo.py`	Seleciona a versão da outra branch para o arquivo inteiro.
`git mergetool`	Abre uma ferramenta de resolução de conflitos.
`git merge --abort`	Cancela o merge em andamento.
`git merge --continue`	Conclui o merge após a resolução dos conflitos.

Git rebase

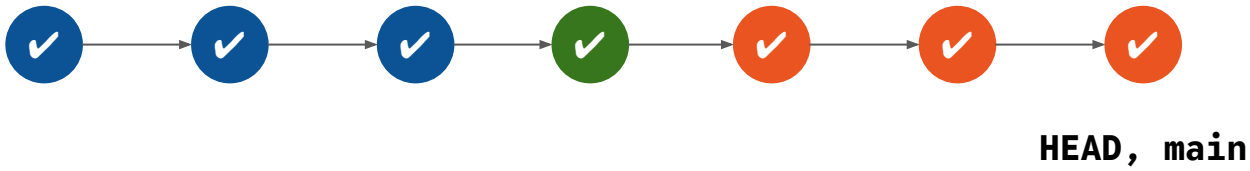
O git rebase reorganiza o histórico de uma branch, replicando seus commits sobre uma nova base, geralmente o commit mais recente de outra branch.




```
git switch feature
git rebase main
```

Git rebase

O git rebase reorganiza o histórico de uma branch, replicando seus commits sobre uma nova base, geralmente o commit mais recente de outra branch.



Git rebase

Importante: **`git rebase main`** não significa trazer os commits de **`main`** para dentro da branch atual por meio de merge.

Significa reconstruir a sequência de commits da branch atual a partir de **`main`**.

```
git switch feature
git rebase main
```

Em caso de dúvidas procure por help

O git provê uma excelente documentação, que é baixada junto com o programa.

Para consultar a página de documentação de um comando basta adicionar o parâmetro **--help**.

```
git branch --help
```

```
$ git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           ...

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone      Clone a repository into a new directory
   init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add        Add file contents to the index
   mv         Move or rename a file, a directory, or a symlink
   restore    Restore working tree files
   rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
   bisect     Use binary search to find the commit that introduced a bug
   diff       Show changes between commits, commit and working tree, etc
   grep       Print lines matching a pattern
   log        Show commit logs
   show       Show various types of objects
   status     Show the working tree status

grow, mark and tweak your common history
   backfill   Download missing objects in a partial clone
   branch     List, create, or delete branches
   commit     Record changes to the repository
   history    EXPERIMENTAL: Rewrite history
   merge      Join two or more development histories together
   rebase     Reapply commits on top of another base tip
   reset      Set `HEAD` or the index to a known state
   switch     Switch branches
   tag        Create, list, delete or verify tags

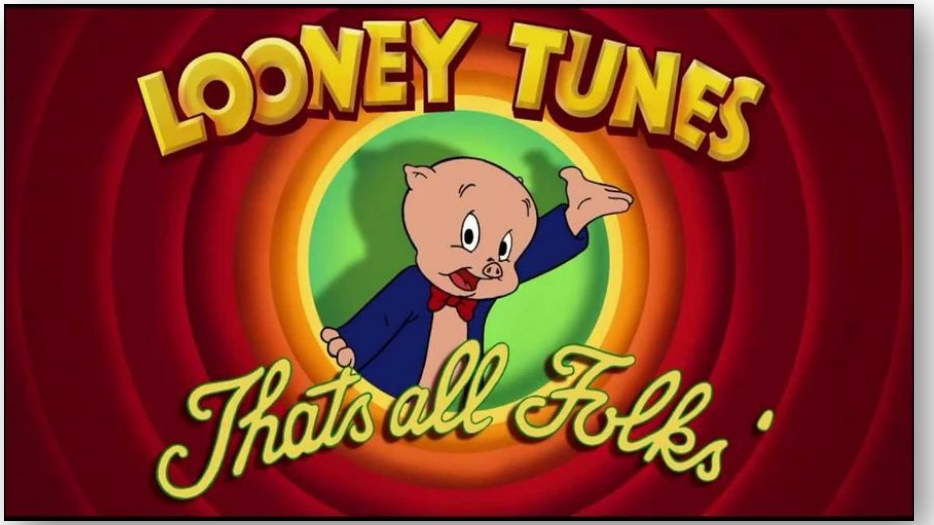
collaborate (see also: git help workflows)
   fetch      Download objects and refs from another repository
   pull       Fetch from and integrate with another repository or a local branch
   push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system
```

Isso é tudo pessoal!!

Existem muitas operações que não cobrimos neste tutorial.

Mas agora você tem uma boa base para continuar os seus estudos!!





Obrigado!

Prof. Giliard Godoi

giliard.godoi@fatecourinhos.edu.br